Kernel Fusion in PyCUDA and PyOpenCL: Layout Conversion with a Per-Channel Affine Transform

Nrushad Joshi
Columbia University

***Abstract*—This project implements a fused layout conversion and per-channel affine transform, $Y[n,c] = \gamma[c] \cdot X[c,n] + \beta[c]$, in both CUDA (via PyCUDA) and OpenCL (via PyOpenCL). Three implementations are compared against a NumPy CPU baseline: two kernels with the intermediate copied through the host, two kernels with the intermediate kept in device global memory, and a single fused kernel. The PyCUDA versions are profiled with NVIDIA Nsight Systems and Nsight Compute on an NVIDIA L4. Fusion cuts kernel time by 1.62× at large N and halves DRAM traffic, closely matching a simple byte-count model. End-to-end, however, host transfers account for over 95% of the total, so the fused and unfused versions finish within about 1% of each other. Profiling also shows that fusion does not fix the transpose's strided store, which runs at 12.5% global store efficiency.**

I. Introduction

Inference pipelines convert tensors between channel-major and batch-major layouts between stages. At inference time a BatchNorm layer reduces to a per-channel affine transform, because the running mean and variance are frozen after training; the trained parameters fold into two per-channel constants, γ and β. The combined operation takes a channel-major input $X \in \mathbb{R}^{C \times N}$ and produces a batch-major output $Y \in \mathbb{R}^{N \times C}$:

$$Y[n,c] = \gamma[c] \cdot X[c,n] + \beta[c]. \tag{1}$$

Kernel fusion is integrating multiple GPU kernels into one kernel to avoid writing and reading intermediate arrays in device global memory between kernels, and to avoid kernel launch overhead. FlashAttention is an example of kernel fusion, where the kernel reduces the memory overhead of attention in transformers by incorporating the QK^T, softmax and V operations into one kernel, so the $N \times N$ attention score matrix is never materialized in GPU memory.

Three implementations of the operation are compared:

- `unfused_hostroundtrip()`: a transpose kernel and an affine kernel, with the intermediate X^T copied back to the host and up again in between.
- `unfused()`: the same two kernels, with X^T kept in device global memory.
- `fused()`: one kernel, with no intermediate array.

Comparing the first two isolates the cost of a host round trip. Comparing the last two isolates the cost of materializing the intermediate in device global memory and launching one extra kernel.

II. Experimental Setup

- GPU: NVIDIA L4 (24 GB GDDR6, 300 GB/s peak)
- Driver 560.35.03, CUDA 12.6
- NSight Systems 2024.4.2.133
- NSight Compute 2024.3.1.0

Every kernel uses a 32×32 block. The transpose and fused kernels map n to `threadIdx.x` and c to `threadIdx.y`; the affine kernel maps c to `threadIdx.x`. Every kernel is boundary-checked, so C and N may take any positive value. In PyCUDA, both kernel-only and end-to-end times are measured with CUDA events; in PyOpenCL, kernel time comes from OpenCL event profiling and end-to-end time from `time.time()`. End-to-end time includes device allocation, the host-to-device copies, the kernel(s) and the device-to-host copy. All three versions are checked against NumPy with `np.allclose(rtol=1e-5, atol=1e-6)`, including edge cases with C or $N = 1$, dimensions that are not multiples of 32, and both extreme aspect ratios.

III. Benchmark Results

A. Benchmark Sweep (Fig. 1)

With $C = 128$ and N swept from 10 to 10^6, the curves split into two regimes.

Small N ($\leq 10^3$). Every GPU total sits between 0.25 and 0.9 ms in PyCUDA and between 0.6 and 1.9 ms in PyOpenCL, almost independent of N. These totals are dominated by fixed costs: device allocation, transfer setup and kernel launch. The CPU baseline is faster than every GPU total in this regime (e.g. 34 μs vs. 246 μs for the fused total at $N = 10$). The kernel-only curves are also flat, since the GPU is mostly idle at these sizes. The first PyCUDA hostroundtrip point (120 μs at $N = 10$) is a warm-up outlier.

Large N ($\geq 10^5$). All curves grow linearly with N. Fusion reduces kernel time by a consistent factor: at $N = 10^6$ the fused kernel takes 6.35 ms against 10.29 ms for the unfused pair in PyCUDA (1.62×), and 6.30 ms against 10.23 ms in PyOpenCL (1.62×). This matches the removal of one $(N \times C)$ write and one $(N \times C)$ read of the intermediate X^T. The two unfused versions have identical kernel times, as expected, since they launch the same two kernels.

The totals tell a different story. `unfused_hostroundtrip()` takes roughly twice as long as `unfused()` (661 ms vs. 341 ms at $N = 10^6$ in PyCUDA), because it adds a DtoH and an HtoD copy of X^T, doubling the PCIe traffic. `unfused()` and `fused()` totals are within about 1% (341 ms vs. 337 ms), because the host transfers, which are identical in both, account for over 95% of the total and the kernel saving is only about 4 ms. For the

TABLE I: Span and Total Memcpy Time at $C = 256$, $N = 65536$

Version	Start (ms)	End (ms)	Span (ms)	Memcpy (ms)
hostroundtrip	431.07	518.51	87.43	81.89
unfused	522.85	565.99	43.13	41.39
fused	570.60	613.79	43.19	42.22

same reason the GPU totals only match the CPU at large N (337 ms vs. 355 ms at $N = 10^6$): the operation does very little arithmetic per byte, so it is bound by PCIe bandwidth once transfers are included.

PyCUDA vs. PyOpenCL. Kernel times agree closely at large N. At small N, the OpenCL kernel times are lower (8 μs vs. 29 μs for the fused kernel at $N = 10$), likely because OpenCL event profiling reports only device execution, while CUDA events also capture some launch latency. OpenCL totals are about 2× the CUDA totals at large N (718 ms vs. 337 ms for fused at $N = 10^6$), which likely reflects slower host-to-device copies through PyOpenCL's buffer-creation path.

B. Aspect-Ratio Sweep (Fig. 2)

The fused kernel's runtime increases slightly as C grows from 32 to 32,768 (0.876 ms to 0.947 ms), but the increase is negligible, and the curve can be considered flat. Since $C \times N$ remains constant at 2^{24}, the total amount of data processed and the number of threads stay the same, and because all dimensions are multiples of 32, every warp is fully populated. Within a warp, consecutive threads differ in n, so the read from $X[c \cdot N + n]$ is always contiguous and coalesced, while the write to $Y[n \cdot C + c]$ is strided by $4C \geq 128$ bytes at every point in the sweep, so each thread's store falls in its own sector. Since this coalescing behaviour is the same at every aspect ratio, changing the aspect ratio does not substantially affect memory access efficiency.

IV. NSight Systems Analysis

A. Launch Structure

`unfused_hostroundtrip()` and `unfused()` each launch two kernels; `fused()` launches one. In `unfused()` at $C = 256$, $N = 65536$, `transpose_kernel` ends at 1537.988 ms and `affine_kernel` starts at 1537.994 ms, giving a dispatch gap of 5.7 μs.

Table I gives, for each version at $C = 256$, $N = 65536$, the timeline span (first HtoD start to last DtoH end, times after 1 s) and the total memcpy time (sum of all HtoD and DtoH bars).

Between `unfused_hostroundtrip()` and `unfused()`, total memcpy time drops from 81.89 ms to 41.39 ms, while the kernel durations stay the same. The removed time is the host round trip of the intermediate X^T: a DtoH copy and an HtoD copy. The remaining difference between `unfused()` and `fused()` cannot come from transfers, since both copy the same bytes. Instead it comes from kernel execution time: `unfused()` still writes X^T to device global memory and reads it back. The `unfused()` timeline shows no memcpy between the two kernel bars, confirming that X^T stays on the device.

TABLE II: Kernel Time vs. Total Time at $C = 256$

N	Version	Kernel (ms)	Total (ms)	Total−Kernel (ms)
256	unfused	0.060	0.528	0.468
	fused	0.038	0.458	0.420
4096	unfused	0.130	2.732	2.602
	fused	0.110	2.560	2.451
65536	unfused	1.406	45.61	44.20
	fused	0.891	44.63	43.74
1048576	unfused	21.40	709.3	687.9
	fused	13.10	701.8	688.7

TABLE III: Launch Call Duration vs. Kernel Run Duration at $C = 256$

Version	Kernel	Call (μs)	Kernel (μs)	Call/Kernel
$N = 4096$				
hostroundtrip	transpose	25.0	53.1	47.2%
hostroundtrip	affine	24.4	11.1	219.5%
unfused	transpose	18.9	53.0	35.6%
unfused	affine	11.8	11.2	106.1%
fused	fused	15.2	53.4	28.5%
$N = 65536$				
hostroundtrip	transpose	25.7	815.1	3.16%
hostroundtrip	affine	27.9	429.2	6.49%
unfused	transpose	25.5	818.3	3.12%
unfused	affine	45.5	514.3	8.85%
fused	fused	29.6	817.6	3.62%

TABLE IV: NSight Compute Metrics at $C = 256$, $N = 65536$ (NVIDIA L4)

Metric	transpose	affine	fused
Duration (ms)	1.71	0.50	1.72
DRAM read (bytes)	80,685,312	73,943,808	80,753,152
DRAM write (bytes)	57,306,112	52,368,384	57,339,776
Store efficiency (%)	12.5	100	12.5
Mem. throughput (GB/s)	–	–	80.48
Load efficiency (%)	–	–	70.8

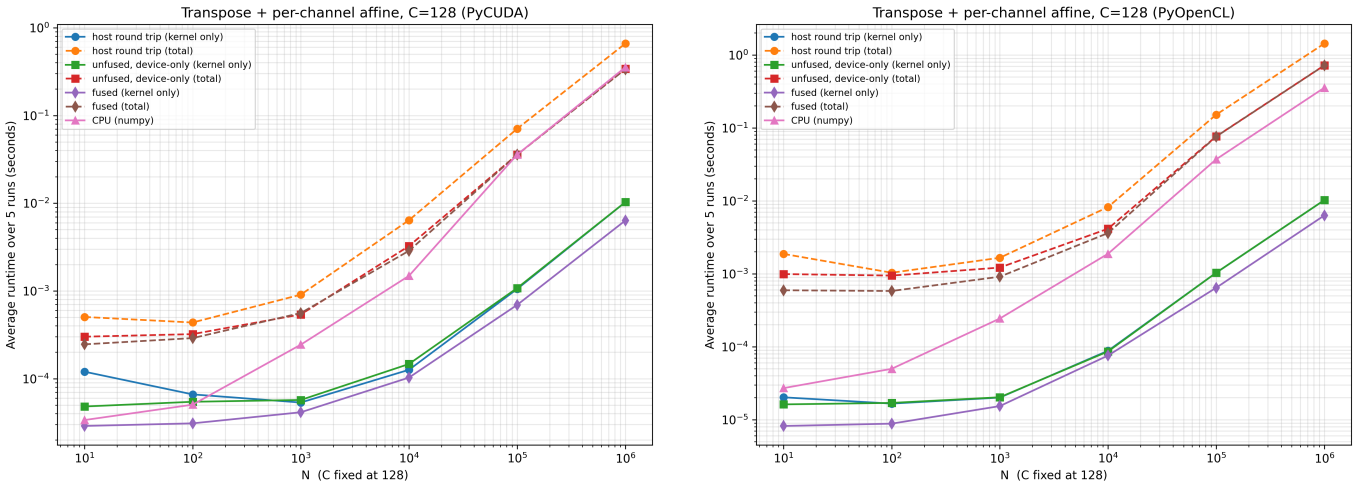
B. Launch-Overhead Crossover

Table II shows that total−kernel time is fixed overhead at small N (about 0.4–0.5 ms, roughly 8–11× the kernel time at $N = 256$) and is dominated by transfers at large N, where the kernel time grows to 1.4 ms (unfused) and 0.9 ms (fused) at $N = 65536$. Table III compares the duration of each kernel launch call (CUDA API row) with the kernel's run duration on the GPU. The launch call takes roughly 12–46 μs at every N, while kernel run time scales with $C \times N$. At $N = 4096$ the launch call is 28–220% of the kernel duration, so per-launch overhead is clearly visible. At $N = 65536$ it falls to 3–9%, so the overhead significantly decreases relative to the kernel duration from $N = 65536$ onward. The crossover exists because launch cost is fixed per kernel while kernel work grows with N. Fusion is therefore most worthwhile at small N, where removing a launch saves a significant fraction of the GPU time.

V. NSight Compute Analysis

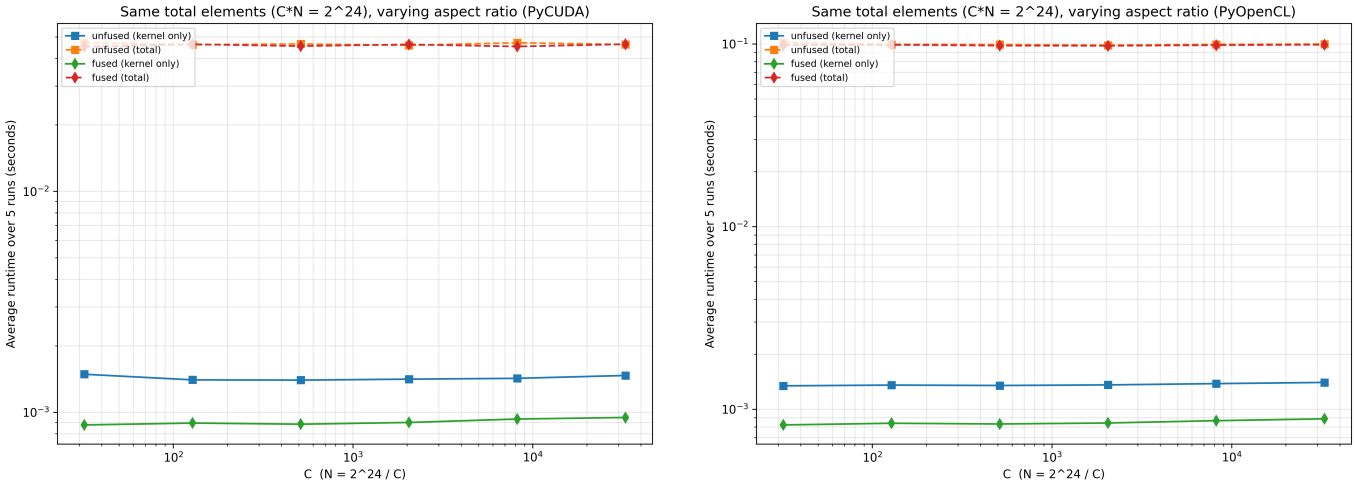
A. Predicted vs. Measured DRAM Bytes

Predicted ($4CN = 67{,}108{,}864$ bytes):



(a) PyCUDA (b) PyOpenCL

Fig. 1: Benchmark sweep: runtime vs. N at $C = 128$.



(a) PyCUDA (b) PyOpenCL

Fig. 2: Aspect-ratio sweep: runtime vs. C with $C \cdot N = 2^{24}$.

- Unfused: $16CN = 268{,}435{,}456$ bytes
- Fused: $8CN = 134{,}217{,}728$ bytes

Measured (`dram__bytes_read.sum` + `dram__bytes_write.sum`):

- Unfused: $137{,}991{,}424 + 126{,}312{,}192 = 264{,}303{,}616$ bytes, ratio 0.985
- Fused: $138{,}092{,}928$ bytes, ratio 1.029

Achieved memory throughput of `fused_kernel`: 80.48 GB/s, which is 26.8% of the L4's 300 GB/s theoretical peak. The measured totals match the predictions closely (within 3%). The small differences come from the L2 cache: some written data is still sitting in L2 when the kernel ends, so it is not counted as a DRAM write, and the strided stores cause some extra DRAM reads to fill partially written sectors. These two effects roughly cancel out.

B. Global Store Efficiency

Global store efficiency is 12.5% for `transpose_kernel`, 100% for `affine_kernel` and 12.5% for `fused_kernel`. Global load efficiency for `fused_kernel` is 70.8%. The read from X is fully coalesced, but the `gamma` and `beta` loads are single 4-byte values shared by the whole warp, and each one still fetches a full 32-byte sector, which lowers the average.

Warp-level explanation. In `fused_kernel` ($n \leftarrow$ `threadIdx.x`, $c \leftarrow$ `threadIdx.y`), consecutive threads in a warp have consecutive n. The read $X[c \cdot N + n]$ has a stride of 4 bytes (coalesced), but the write $Y[n \cdot C + c]$ has a stride of $4C = 1024$ bytes, so each thread writes to its own 32-byte sector and uses only 4 bytes of it (12.5%). `transpose_kernel` has the same pattern.

In `affine_kernel` ($c \leftarrow$ `threadIdx.x`), consecutive threads have consecutive c, so both the read $X^T[n \cdot C + c]$ and the write $Y[n \cdot C + c]$ have a stride of 4 bytes (100%).

The mappings differ because `affine_kernel` reads and writes arrays with the same (N, C) layout, so one mapping makes both accesses contiguous. The transpose and fused kernels read a (C, N) array and write an (N, C) array, so only one side can be contiguous: the read is kept coalesced and the write is strided.

Improving store efficiency. To make the stores coalesced, the transpose has to happen in shared memory instead of global memory. In a shared-memory tiled transpose, each 32×32 block loads a tile of X with coalesced reads into shared memory, synchronizes, and then writes the tile to Y with the indices swapped, so that consecutive threads write consecutive addresses. Both the global reads and the global writes are then coalesced, and the strided access happens in fast on-chip shared memory.

What fusion changes. Fusion removes the intermediate X^T and its extra write and read, but it does not change the transpose's strided, uncoalesced write pattern.

VI. Conclusion

Fusing the transpose and the affine transform into one kernel removes the intermediate array, halves DRAM traffic and cuts kernel time by $1.62\times$ at large N. Removing a kernel launch also matters at small N, where launch overhead is a large fraction of GPU time. End-to-end, however, the operation is bound by PCIe transfers, so the kernel saving is hidden behind them, and the strided store into Y remains the main inefficiency inside the kernel. A shared-memory tiled transpose is the natural next step.

Acknowledgment

This work was completed as a homework assignment for EECS E4750: Heterogeneous Computing for Signal and Data Processing at Columbia University, taught by Prof. Zoran Kostić. The assignment template, test harness, and benchmarking code were provided by the course. The CUDA and OpenCL kernels and all host code were developed by the author. This report was written with the assistance of AI; all experimental results were produced and verified by the author.